```
        v-for="module in filteredModules"
        :key="module.id"
        class="module-group"
      >
        <button
          type="button"
          class="module-row"
          :class="{ active: module.id === selectedModuleId }"
          :aria-expanded="isExpanded(module.id)"
          @click="toggleModule(module)"
        >
          <ChevronRight
            class="chevron"
            :class="{ expanded: isExpanded(module.id) }"
            :size="16"
          />
          <Layers3 :size="16" />
          <span class="module-name">{{ module.name }}</span>
          <span class="item-count">{{ module.knowledgePoints.length }}</span>
          <LoaderCircle
            v-if="loadingModuleId === module.id"
            class="spin"
            :size="14"
          />
        </button>
        <div
          v-if="isExpanded(module.id)"
          class="knowledge-list"
        >
          <button
            v-for="knowledge in module.knowledgePoints"
            :key="knowledge.id"
            type="button"
            class="knowledge-row"
            :class="{ active: knowledge.id === selectedKnowledgePointId }"
            @click="selectKnowledge(module.id, knowledge.id)"
          >
            <FileText :size="14" />
            <span>{{ knowledge.name }}</span>
          </button>
        </div>
      </div>
    </nav>
  </aside>
</template>
<script setup>
import { computed, ref, watch } from 'vue'
import {
  ChevronRight,
  Database,
```

```
  FileText,
  Layers3,
  LoaderCircle,
  Search,
} from '@lucide/vue'
const props = defineProps({
  modules: { type: Array, default: () => [] },
  selectedModuleId: { type: String, default: '' },
  selectedKnowledgePointId: { type: String, default: '' },
  loading: { type: Boolean, default: false },
  loadingModuleId: { type: String, default: '' },
})
const emit = defineEmits(['select-module', 'select-knowledge'])
const keyword = ref('')
const expandedModuleId = ref('')
const knowledgePointCount = computed(() =>
  props.modules.reduce(
    (total, module) => total + module.knowledgePoints.length,
    0,
  ),
)
const filteredModules = computed(() => {
  const query = keyword.value.toLowerCase()
  if (!query) return props.modules
  return props.modules
    .map((module) => {
      const moduleMatches = module.name.toLowerCase().includes(query)
      const matchingKnowledgePoints = module.knowledgePoints.filter(
        (knowledge) => knowledge.name.toLowerCase().includes(query),
      )
      if (moduleMatches) return module
      if (matchingKnowledgePoints.length) {
        return { ...module, knowledgePoints: matchingKnowledgePoints }
      }
      return null
    })
    .filter(Boolean)
})
watch(
  () => props.selectedModuleId,
  (moduleId) => {
    if (moduleId) expandedModuleId.value = moduleId
  },
)
function isExpanded(moduleId) {
  return Boolean(keyword.value) || expandedModuleId.value === moduleId
}
function toggleModule(module) {
  expandedModuleId.value =
    expandedModuleId.value === module.id ? '' : module.id
```

```
  emit('select-module', module.id)
}
function selectKnowledge(moduleId, knowledgePointId) {
  emit('select-knowledge', { moduleId, knowledgePointId })
}
</script>
<style scoped>
.directory-panel {
  height: 100%;
  min-height: 0;
  padding: 18px 14px;
  border-right: 1px solid #e2e8f0;
  background: #ffffff;
  display: flex;
  flex-direction: column;
  overflow: hidden;
}
.panel-heading {
  display: flex;
  align-items: center;
  gap: 10px;
  padding: 0 4px;
}
.heading-icon {
  width: 34px;
  height: 34px;
  border-radius: 7px;
  display: grid;
  place-items: center;
  color: #ffffff;
  background: #7c3aed;
}
.eyebrow {
  display: block;
  margin-bottom: 2px;
  color: #64748b;
  font-size: 11px;
}
h2 {
  margin: 0;
  color: #0f172a;
  font-size: 16px;
  line-height: 1.2;
}
.directory-summary {
  display: flex;
  justify-content: space-between;
  margin: 14px 4px 12px;
  color: #64748b;
  font-size: 12px;
}
```

```
.search-box {
  height: 38px;
  padding: 0 10px;
  border: 1px solid #d7dee8;
  border-radius: 6px;
  display: flex;
  align-items: center;
  gap: 8px;
  color: #64748b;
  background: #f8fafc;
}
.search-box:focus-within {
  border-color: #7c3aed;
  box-shadow: 0 0 0 3px rgba(124, 58, 237, 0.1);
  background: #ffffff;
}
.search-box input {
  width: 100%;
  min-width: 0;
  border: 0;
  outline: 0;
  color: #0f172a;
  background: transparent;
  font-size: 13px;
}
.directory-tree {
  min-height: 0;
  margin-top: 10px;
  padding-right: 2px;
  overflow-y: auto;
}
.module-group + .module-group {
  margin-top: 3px;
}
.module-row,
.knowledge-row {
  width: 100%;
  border: 0;
  display: flex;
  align-items: center;
  color: #334155;
  background: transparent;
  cursor: pointer;
  text-align: left;
}
.module-row {
  min-height: 38px;
  padding: 7px 8px;
  border-radius: 6px;
  gap: 7px;
}
```

```
.module-row:hover,
.module-row.active {
  color: #5b21b6;
  background: #f3e8ff;
}
.chevron {
  flex: 0 0 auto;
  transition: transform 0.18s ease;
}
.chevron.expanded {
  transform: rotate(90deg);
}
.module-name {
  min-width: 0;
  flex: 1;
  overflow: hidden;
  font-size: 13px;
  font-weight: 600;
  text-overflow: ellipsis;
  white-space: nowrap;
}
.item-count {
  flex: 0 0 auto;
  color: #64748b;
  font-size: 11px;
}
.knowledge-list {
  margin: 2px 0 6px 24px;
  border-left: 1px solid #e2e8f0;
}
.knowledge-row {
  min-height: 34px;
  padding: 6px 8px;
  border-radius: 0 6px 6px 0;
  gap: 7px;
  font-size: 12px;
  line-height: 1.35;
}
.knowledge-row:hover,
.knowledge-row.active {
  color: #1d4ed8;
  background: #eff6ff;
}
.knowledge-row span {
  min-width: 0;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}
.panel-state {
  flex: 1;
```

```
  min-height: 160px;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: 8px;
  color: #64748b;
  font-size: 13px;
}
.spin {
  animation: spin 0.9s linear infinite;
}
@keyframes spin {
  to {
    transform: rotate(360deg);
  }
}
</style>
================================================================================================
FILE: frontend/src/components/KnowledgeGraph.vue
================================================================================================
<template>
  <div ref="graphContainer" class="graph-container">
    <svg ref="svgRef" aria-label="知识图谱可视化"></svg>
    <div v-if="loading" class="graph-state">
      <LoaderCircle class="spin" :size="24" />
      <span>正在加载图谱</span>
    </div>
    <div v-else-if="nodes.length === 0" class="graph-state">
      <Network :size="28" />
      <strong>暂无可展示内容</strong>
      <span>请从左侧目录选择模块</span>
    </div>
    <div class="graph-controls">
      <button
        type="button"
        title="放大"
        aria-label="放大图谱"
        @click="zoomBy(1.25)"
      >
        <ZoomIn :size="16" />
      </button>
      <button
        type="button"
        title="缩小"
        aria-label="缩小图谱"
        @click="zoomBy(0.8)"
      >
        <ZoomOut :size="16" />
      </button>
      <button
```

```
        type="button"
        title="适应画布"
        aria-label="适应画布"
        @click="fitGraph(true)"
      >
        <Maximize2 :size="16" />
      </button>
    </div>
    <div class="graph-legend">
      <span><i class="legend-module"></i>模块</span>
      <span><i class="legend-knowledge"></i>知识点</span>
    </div>
  </div>
</template>
<script setup>
import {
  nextTick,
  onBeforeUnmount,
  onMounted,
  ref,
  watch,
} from 'vue'
import * as d3 from 'd3'
import {
  LoaderCircle,
  Maximize2,
  Network,
  ZoomIn,
  ZoomOut,
} from '@lucide/vue'
import {
  getNodeName,
  getNodeType,
  NODE_TYPE_COLORS,
  RELATION_LABELS,
} from '../constants/graph'
const props = defineProps({
  nodes: { type: Array, default: () => [] },
  links: { type: Array, default: () => [] },
  selectedId: { type: String, default: '' },
  loading: { type: Boolean, default: false },
})
const emit = defineEmits(['node-click'])
const graphContainer = ref(null)
const svgRef = ref(null)
let simulation = null
let resizeObserver = null
let renderTimer = null
let fitTimer = null
let zoomBehavior = null
let svgSelection = null
```

```
let graphSelection = null
let nodeSelection = null
function nodeRadius(node) {
  return getNodeType(node.labels) === 'Module' ? 21 : 10
}
function clearTimers() {
  if (renderTimer) {
    clearTimeout(renderTimer)
    renderTimer = null
  }
  if (fitTimer) {
    clearTimeout(fitTimer)
    fitTimer = null
  }
}
function stopSimulation() {
  if (simulation) {
    simulation.stop()
    simulation = null
  }
}
function clearGraph() {
  stopSimulation()
  if (svgSelection) {
    svgSelection.selectAll('*').remove()
  }
  nodeSelection = null
  graphSelection = null
}
function truncateLabel(value, maxLength = 12) {
  const text = String(value || '')
  return text.length > maxLength
    ? `${text.slice(0, maxLength)}...`
    : text
}
function updateSelection() {
  if (!nodeSelection) return
  nodeSelection
    .attr('stroke', (node) =>
      node.id === props.selectedId ? '#0f172a' : '#ffffff',
    )
    .attr('stroke-width', (node) =>
      node.id === props.selectedId ? 3 : 2,
    )
}
function renderGraph() {
  clearTimers()
  stopSimulation()
  if (!svgRef.value || !graphContainer.value) return
  const width = graphContainer.value.clientWidth
  const height = graphContainer.value.clientHeight
```

```
svgSelection = d3.select(svgRef.value)
clearGraph()
svgSelection = d3.select(svgRef.value)
if (!width || !height) return
svgSelection
  .attr('width', width)
  .attr('height', height)
  .attr('viewBox', `0 0 ${width} ${height}`)
if (!props.nodes.length) return
const nodes = props.nodes.map((node) => ({
  ...node,
  id: String(node.id),
}))
const links = props.links
  .filter((link) => {
    const nodeIds = new Set(nodes.map((node) => node.id))
    return nodeIds.has(String(link.source)) && nodeIds.has(String(link.target))
  })
  .map((link) => ({
    ...link,
    source: String(link.source),
    target: String(link.target),
  }))
const definitions = svgSelection.append('defs')
definitions
  .append('marker')
  .attr('id', 'graph-arrow')
  .attr('viewBox', '0 -5 10 10')
  .attr('refX', 10)
  .attr('refY', 0)
  .attr('markerWidth', 6)
  .attr('markerHeight', 6)
  .attr('orient', 'auto')
  .append('path')
  .attr('d', 'M0,-5L10,0L0,5')
  .attr('fill', '#94a3b8')
zoomBehavior = d3
  .zoom()
  .scaleExtent([0.25, 4])
  .on('zoom', (event) => {
    graphSelection.attr('transform', event.transform)
  })
svgSelection.call(zoomBehavior)
graphSelection = svgSelection.append('g')
const link = graphSelection
  .append('g')
  .attr('class', 'graph-links')
  .selectAll('line')
  .data(links)
  .enter()
  .append('line')
```

```
  .attr('stroke', '#94a3b8')
  .attr('stroke-opacity', 0.5)
  .attr('stroke-width', (item) =>
    item.type === 'BELONGS_TO' ? 1.2 : 1.5,
  )
  .attr('marker-end', 'url(#graph-arrow)')
link
  .append('title')
  .text((item) => RELATION_LABELS[item.type] || item.type)
nodeSelection = graphSelection
  .append('g')
  .attr('class', 'graph-nodes')
  .selectAll('circle')
  .data(nodes)
  .enter()
  .append('circle')
  .attr('class', 'graph-node')
  .attr('r', nodeRadius)
  .attr('fill', (node) => NODE_TYPE_COLORS[getNodeType(node.labels)])
  .attr('stroke', '#ffffff')
  .attr('stroke-width', 2)
  .attr('cursor', 'pointer')
  .call(
    d3
      .drag()
      .on('start', dragStarted)
      .on('drag', dragged)
      .on('end', dragEnded),
  )
  .on('click', (event, node) => {
    event.stopPropagation()
    emit('node-click', node)
  })
nodeSelection
  .append('title')
  .text((node) => {
    const type = getNodeType(node.labels)
    return `${type === 'Module' ? '模块' : '知识点'}：${getNodeName(node)}`
  })
const label = graphSelection
  .append('g')
  .attr('class', 'graph-labels')
  .selectAll('text')
  .data(nodes)
  .enter()
  .append('text')
  .text((node) => truncateLabel(getNodeName(node)))
  .attr('font-size', (node) =>
    getNodeType(node.labels) === 'Module' ? 12 : 10,
  )
  .attr('font-weight', (node) =>
```

```
    getNodeType(node.labels) === 'Module' ? 700 : 500,
  )
  .attr('text-anchor', 'middle')
  .attr('dy', (node) => (getNodeType(node.labels) === 'Module' ? -29 : -17))
  .attr('fill', '#334155')
  .style('paint-order', 'stroke')
  .style('stroke', '#ffffff')
  .style('stroke-width', 3)
  .style('stroke-linejoin', 'round')
  .style('user-select', 'none')
  .style('pointer-events', 'none')
updateSelection()
simulation = d3
  .forceSimulation(nodes)
  .force(
    'link',
    d3
      .forceLink(links)
      .id((node) => node.id)
      .distance((item) =>
        item.type === 'BELONGS_TO' ? 74 : 108,
      )
      .strength((item) =>
        item.type === 'BELONGS_TO' ? 0.9 : 0.55,
      ),
  )
  .force(
    'charge',
    d3.forceManyBody().strength((node) =>
      getNodeType(node.labels) === 'Module' ? -520 : -190,
    ),
  )
  .force('center', d3.forceCenter(width / 2, height / 2))
  .force('x', d3.forceX(width / 2).strength(0.025))
  .force('y', d3.forceY(height / 2).strength(0.035))
  .force(
    'collision',
    d3
      .forceCollide()
      .radius((node) => nodeRadius(node) + 30)
      .iterations(2),
  )
  .on('tick', () => {
    nodes.forEach((node) => {
      const padding = nodeRadius(node) + 18
      node.x = Math.max(padding, Math.min(width - padding, node.x))
      node.y = Math.max(padding, Math.min(height - padding, node.y))
    })
    link
      .attr('x1', (item) => item.source.x)
      .attr('y1', (item) => item.source.y)
```

```
        .attr('x2', (item) => item.target.x)
        .attr('y2', (item) => item.target.y)
      nodeSelection
        .attr('cx', (node) => node.x)
        .attr('cy', (node) => node.y)
      label
        .attr('x', (node) => node.x)
        .attr('y', (node) => node.y)
    })
  fitTimer = setTimeout(() => fitGraph(false), 650)
  function dragStarted(event, node) {
    if (!event.active) simulation.alphaTarget(0.25).restart()
    node.fx = node.x
    node.fy = node.y
  }
  function dragged(event, node) {
    node.fx = event.x
    node.fy = event.y
  }
  function dragEnded(event, node) {
    if (!event.active) simulation.alphaTarget(0)
    node.fx = null
    node.fy = null
  }
}
function fitGraph(animate = true) {
  if (!graphSelection || !svgSelection || !zoomBehavior) return
  if (!graphSelection.node()) return
  const width = graphContainer.value?.clientWidth || 0
  const height = graphContainer.value?.clientHeight || 0
  if (!width || !height) return
  const bounds = graphSelection.node().getBBox()
  if (!bounds.width || !bounds.height) return
  const scale = Math.max(
    0.3,
    Math.min(
      2,
      Math.min(width / bounds.width, height / bounds.height) * 0.82,
    ),
  )
  const midX = bounds.x + bounds.width / 2
  const midY = bounds.y + bounds.height / 2
  const transform = d3.zoomIdentity
    .translate(width / 2, height / 2)
    .scale(scale)
    .translate(-midX, -midY)
  const target = animate ? svgSelection.transition().duration(450) : svgSelection
  target.call(zoomBehavior.transform, transform)
}
function zoomBy(factor) {
  if (!svgSelection || !zoomBehavior) return
```

```
  svgSelection
    .transition()
    .duration(180)
    .call(zoomBehavior.scaleBy, factor)
}
function scheduleRender(delay = 80) {
  nextTick(() => {
    if (renderTimer) clearTimeout(renderTimer)
    renderTimer = setTimeout(renderGraph, delay)
  })
}
watch(
  () => [props.nodes, props.links],
  () => scheduleRender(),
  { flush: 'post' },
)
watch(
  () => props.selectedId,
  () => updateSelection(),
)
onMounted(() => {
  scheduleRender(120)
  resizeObserver = new ResizeObserver(() => scheduleRender(120))
  if (graphContainer.value) {
    resizeObserver.observe(graphContainer.value)
  }
})
onBeforeUnmount(() => {
  clearTimers()
  stopSimulation()
  if (resizeObserver) {
    resizeObserver.disconnect()
    resizeObserver = null
  }
})
</script>
<style scoped>
.graph-container {
  position: relative;
  width: 100%;
  height: 100%;
  min-height: 0;
  overflow: hidden;
  background:
    linear-gradient(rgba(148, 163, 184, 0.09) 1px, transparent 1px),
    linear-gradient(90deg, rgba(148, 163, 184, 0.09) 1px, transparent 1px),
    #ffffff;
  background-size: 24px 24px;
}
.graph-container svg {
  display: block;
```

```
  width: 100%;
  height: 100%;
}
.graph-state {
  position: absolute;
  inset: 0;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: 8px;
  color: #64748b;
  background: rgba(255, 255, 255, 0.82);
  font-size: 13px;
  text-align: center;
}
.graph-state strong {
  color: #334155;
  font-size: 14px;
}
.graph-controls {
  position: absolute;
  top: 14px;
  left: 14px;
  display: flex;
  flex-direction: column;
  gap: 6px;
}
.graph-controls button {
  width: 34px;
  height: 34px;
  padding: 0;
  border: 1px solid #dbe4ef;
  border-radius: 6px;
  display: grid;
  place-items: center;
  color: #475569;
  background: rgba(255, 255, 255, 0.94);
  box-shadow: 0 4px 12px rgba(15, 23, 42, 0.08);
  cursor: pointer;
}
.graph-controls button:hover {
  color: #1d4ed8;
  border-color: #93c5fd;
}
.graph-legend {
  position: absolute;
  left: 14px;
  bottom: 14px;
  padding: 7px 9px;
  border: 1px solid #e2e8f0;
```

```
  border-radius: 6px;
  display: flex;
  gap: 12px;
  color: #475569;
  background: rgba(255, 255, 255, 0.92);
  font-size: 11px;
}
.graph-legend span {
  display: flex;
  align-items: center;
  gap: 5px;
}
.graph-legend i {
  width: 9px;
  height: 9px;
  border-radius: 50%;
}
.legend-module {
  background: #7c3aed;
}
.legend-knowledge {
  background: #2563eb;
}
.spin {
  animation: spin 0.9s linear infinite;
}
@keyframes spin {
  to {
    transform: rotate(360deg);
  }
}
</style>
================================================================================================
FILE: frontend/src/components/KnowledgeDetail.vue
================================================================================================
<template>
  <aside class="detail-panel" aria-label="知识点详情">
    <div v-if="loading" class="detail-state">
      <LoaderCircle class="spin" :size="22" />
      <span>正在加载知识点</span>
    </div>
    <template v-else-if="detail">
      <header class="detail-header">
        <div class="title-wrap">
          <span class="type-badge">知识点</span>
          <h2>{{ nodeName }}</h2>
        </div>
        <button
          type="button"
          class="icon-button"
          title="关闭详情"
```

```
    aria-label="关闭详情"
    @click="$emit('close')"
  >
    <X :size="18" />
  </button>
</header>
<div class="detail-content">
  <div class="meta-grid">
    <div class="meta-item">
      <Gauge :size="16" />
      <span>难度</span>
      <strong>{{ properties.difficulty || '未标注' }}</strong>
    </div>
    <div class="meta-item">
      <Star :size="16" />
      <span>重要性</span>
      <strong>{{ properties.importance || '未标注' }}</strong>
    </div>
    <div class="meta-item">
      <Tags :size="16" />
      <span>分类</span>
      <strong>{{ properties.subcategory || '未分类' }}</strong>
    </div>
    <div class="meta-item">
      <Network :size="16" />
      <span>所属模块</span>
      <strong>{{ moduleName }}</strong>
    </div>
  </div>
  <section class="content-section">
    <h3><Info :size="15" />知识点说明</h3>
    <p>{{ properties.description || '暂无详细说明。' }}</p>
  </section>
  <section
    v-if="relationGroups.length"
    class="content-section"
  >
    <h3><Link2 :size="15" />关联知识</h3>
    <div
      v-for="group in relationGroups"
      :key="group.type"
      class="relation-group"
    >
      <span class="relation-label">{{ group.label }}</span>
      <button
        v-for="item in group.items"
        :key="item.node.id"
        type="button"
        class="related-row"
        @click="navigateToNode(item.node)"
      >
```

```
      <span>{{ getNodeName(item.node) }}</span>
      <ChevronRight :size="14" />
    </button>
  </div>
</section>
<section class="content-section">
  <h3><PlayCircle :size="15" />教学资源</h3>
  <div v-if="videos.length" class="media-list">
    <article
      v-for="video in videos"
      :key="video.id"
      class="media-card"
    >
      <div class="media-title">
        <Film :size="16" />
        <strong>{{ video.title }}</strong>
      </div>
      <video
        v-if="video.url"
        class="video-player"
        controls
        preload="metadata"
        :poster="video.coverUrl || undefined"
        :src="video.url"
      >
        当前浏览器不支持视频播放。
      </video>
    </article>
  </div>
  <div v-else class="resource-placeholder">
    <Film :size="20" />
    <div>
      <strong>暂无视频资源</strong>
      <span>后续可在媒体清单中添加 MP4 或 WebM 文件。</span>
    </div>
  </div>
  <div v-if="slides.length" class="media-list">
    <a
      v-for="slide in slides"
      :key="slide.id"
      class="media-link"
      :href="slide.url"
      target="_blank"
      rel="noopener"
    >
      <Presentation :size="17" />
      <span>
        <strong>{{ slide.title }}</strong>
        <small>{{ slide.format || '课件' }}</small>
      </span>
      <ExternalLink :size="14" />
```

```
            </a>
          </div>
          <div v-else class="resource-placeholder">
            <Presentation :size="20" />
            <div>
              <strong>暂无课件资源</strong>
              <span>后续可添加 PDF、PPT 或 PPTX 文件。</span>
            </div>
          </div>
        </section>
      </div>
    </template>
  </aside>
</template>
<script setup>
import { computed } from 'vue'
import {
  ChevronRight,
  ExternalLink,
  Film,
  Gauge,
  Info,
  Link2,
  LoaderCircle,
  Network,
  PlayCircle,
  Presentation,
  Star,
  Tags,
  X,
} from '@lucide/vue'
import {
  getNodeName,
  getNodeType,
  RELATION_LABELS,
} from '../constants/graph'
const props = defineProps({
  detail: { type: Object, default: null },
  loading: { type: Boolean, default: false },
})
const emit = defineEmits(['close', 'navigate'])
const properties = computed(() => props.detail?.node?.properties || {})
const nodeName = computed(() => getNodeName(props.detail?.node))
const moduleName = computed(() =>
  props.detail?.module ? getNodeName(props.detail.module) : '未关联模块',
)
const videos = computed(() => props.detail?.materials?.videos || [])
const slides = computed(() => props.detail?.materials?.slides || [])
const relationGroups = computed(() => {
  const groups = new Map()
  for (const relation of props.detail?.relations || []) {
```

```
    if (!groups.has(relation.type)) {
      groups.set(relation.type, {
        type: relation.type,
        label: RELATION_LABELS[relation.type] || relation.type,
        items: [],
      })
    }
    groups.get(relation.type).items.push(relation)
  }
  return Array.from(groups.values())
})
function navigateToNode(node) {
  const type = getNodeType(node.labels)
  if (type === 'Module') {
    emit('navigate', { type, id: node.id })
  } else if (type === 'KnowledgePoint') {
    emit('navigate', {
      type,
      id: node.id,
      moduleId: node.properties.module_id,
    })
  }
}
</script>
<style scoped>
.detail-panel {
  position: absolute;
  z-index: 8;
  top: 14px;
  right: 14px;
  bottom: 14px;
  width: min(410px, calc(100% - 28px));
  border: 1px solid #dfe7f1;
  border-radius: 8px;
  background: #ffffff;
  box-shadow: 0 18px 50px rgba(15, 23, 42, 0.16);
  display: flex;
  flex-direction: column;
  overflow: hidden;
}
.detail-header {
  padding: 18px 18px 14px;
  border-bottom: 1px solid #e7edf5;
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  gap: 12px;
}
.title-wrap {
  min-width: 0;
}
```

```
.type-badge {
  display: inline-flex;
  padding: 3px 7px;
  border-radius: 4px;
  color: #1d4ed8;
  background: #eff6ff;
  font-size: 11px;
  font-weight: 700;
}
h2 {
  margin: 8px 0 0;
  color: #0f172a;
  font-size: 17px;
  line-height: 1.45;
}
.icon-button {
  width: 32px;
  height: 32px;
  border: 0;
  border-radius: 6px;
  display: grid;
  flex: 0 0 auto;
  place-items: center;
  color: #475569;
  background: #f8fafc;
  cursor: pointer;
}
.icon-button:hover {
  color: #0f172a;
  background: #eef2f7;
}
.detail-content {
  min-height: 0;
  padding: 16px 18px 24px;
  overflow-y: auto;
}
.meta-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 8px;
}
.meta-item {
  min-width: 0;
  padding: 10px;
  border: 1px solid #e7edf5;
  border-radius: 6px;
  display: grid;
  grid-template-columns: 18px 1fr;
  column-gap: 6px;
  color: #64748b;
  background: #f8fafc;
```

```
  font-size: 11px;
}
.meta-item svg {
  grid-row: span 2;
  margin-top: 2px;
  color: #2563eb;
}
.meta-item strong {
  min-width: 0;
  overflow: hidden;
  color: #1e293b;
  font-size: 12px;
  text-overflow: ellipsis;
  white-space: nowrap;
}
.content-section + .content-section {
  margin-top: 20px;
}
.content-section h3 {
  margin: 0 0 10px;
  display: flex;
  align-items: center;
  gap: 6px;
  color: #0f172a;
  font-size: 13px;
}
.content-section p {
  margin: 0;
  color: #475569;
  font-size: 13px;
  line-height: 1.75;
}
.relation-group + .relation-group {
  margin-top: 12px;
}
.relation-label {
  display: block;
  margin-bottom: 5px;
  color: #64748b;
  font-size: 11px;
}
.related-row {
  width: 100%;
  padding: 8px 9px;
  border: 0;
  border-radius: 5px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 8px;
  color: #334155;
```

```
  background: #f8fafc;
  cursor: pointer;
  text-align: left;
  font-size: 12px;
}
.related-row + .related-row {
  margin-top: 4px;
}
.related-row:hover {
  color: #1d4ed8;
  background: #eff6ff;
}
.related-row span {
  min-width: 0;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}
.resource-placeholder {
  padding: 11px;
  border: 1px dashed #cbd5e1;
  border-radius: 6px;
  display: flex;
  align-items: center;
  gap: 10px;
  color: #64748b;
  background: #fbfdff;
}
.resource-placeholder + .resource-placeholder {
  margin-top: 8px;
}
.media-list + .media-list,
.resource-placeholder + .media-list,
.media-list + .resource-placeholder {
  margin-top: 8px;
}
.media-card {
  padding: 10px;
  border: 1px solid #e2e8f0;
  border-radius: 6px;
  background: #f8fafc;
}
.media-card + .media-card {
  margin-top: 8px;
}
.media-title {
  margin-bottom: 8px;
  display: flex;
  align-items: center;
  gap: 7px;
  color: #334155;
```

```
  font-size: 12px;
}
.media-title svg {
  color: #7c3aed;
}
.video-player {
  display: block;
  width: 100%;
  max-height: 220px;
  border-radius: 5px;
  background: #0f172a;
}
.media-link {
  padding: 10px;
  border: 1px solid #dbe4ef;
  border-radius: 6px;
  display: flex;
  align-items: center;
  gap: 9px;
  color: #334155;
  background: #ffffff;
  text-decoration: none;
}
.media-link + .media-link {
  margin-top: 6px;
}
.media-link:hover {
  border-color: #93c5fd;
  color: #1d4ed8;
  background: #eff6ff;
}
.media-link > svg:first-child {
  flex: 0 0 auto;
  color: #7c3aed;
}
.media-link > svg:last-child {
  flex: 0 0 auto;
  color: #64748b;
}
.media-link span {
  min-width: 0;
  flex: 1;
}
.media-link strong,
.media-link small {
  display: block;
}
.media-link strong {
  overflow: hidden;
  font-size: 12px;
  text-overflow: ellipsis;
```

```
  white-space: nowrap;
}
.media-link small {
  margin-top: 2px;
  color: #64748b;
  font-size: 10px;
  text-transform: uppercase;
}
.resource-placeholder > svg {
  flex: 0 0 auto;
  color: #7c3aed;
}
.resource-placeholder div {
  min-width: 0;
}
.resource-placeholder strong,
.resource-placeholder span {
  display: block;
}
.resource-placeholder strong {
  margin-bottom: 2px;
  color: #334155;
  font-size: 12px;
}
.resource-placeholder span {
  font-size: 11px;
  line-height: 1.5;
}
.detail-state {
  flex: 1;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: 8px;
  color: #64748b;
  font-size: 13px;
}
.spin {
  animation: spin 0.9s linear infinite;
}
@keyframes spin {
  to {
    transform: rotate(360deg);
  }
}
@media (max-width: 640px) {
  .detail-panel {
    top: 8px;
    right: 8px;
    bottom: 8px;
```

```
    width: calc(100% - 16px);
  }
  .meta-grid {
    grid-template-columns: 1fr;
  }
}
</style>
================================================================================================
FILE: frontend/src/components/ChatPanel.vue
================================================================================================
<template>
  <aside class="chat-panel">
    <header class="chat-header">
      <div class="chat-heading">
        <div class="bot-icon">
          <Bot :size="18" />
        </div>
        <div>
          <h2>智能问答</h2>
          <span>基于知识图谱的课程助手</span>
        </div>
      </div>
      <span
        class="status-dot"
        :class="{ ready: configured }"
      >
        {{ configured ? '已连接' : '待配置' }}
      </span>
    </header>
    <div ref="messageList" class="message-list">
      <div
        v-for="(message, index) in messages"
        :key="`${message.role}-${index}`"
        class="message-row"
        :class="message.role"
      >
        <div class="message-avatar">
          <User v-if="message.role === 'user'" :size="15" />
          <Bot v-else :size="15" />
        </div>
        <div class="message-content">
          <div class="message-bubble">{{ message.content }}</div>
          <div
            v-if="message.role === 'assistant' && message.references?.length"
            class="message-references"
          >
            <button
              v-for="reference in message.references"
              :key="reference.id"
              type="button"
              @click="$emit('select-reference', reference)"
```

```
            >
              {{ reference.name }}
            </button>
          </div>
        </div>
      </div>
      <div v-if="loading" class="message-row assistant">
        <div class="message-avatar">
          <Bot :size="15" />
        </div>
        <div class="message-bubble typing">
          <span></span>
          <span></span>
          <span></span>
        </div>
      </div>
    </div>
    <div class="quick-prompts">
      <button
        v-for="prompt in quickPrompts"
        :key="prompt"
        type="button"
        @click="sendPrompt(prompt)"
      >
        {{ prompt }}
      </button>
    </div>
    <form class="chat-composer" @submit.prevent="submit">
      <textarea
        v-model.trim="draft"
        rows="3"
        placeholder="输入关于知识图谱的问题"
        aria-label="智能问答输入框"
        @keydown.enter.exact.prevent="submit"
      ></textarea>
      <button
        type="submit"
        class="send-button"
        :disabled="!draft || loading"
        title="发送问题"
        aria-label="发送问题"
      >
        <Send :size="17" />
      </button>
    </form>
  </aside>
</template>
<script setup>
import { nextTick, ref, watch } from 'vue'
import { Bot, Send, User } from '@lucide/vue'
const props = defineProps({
```

```
  messages: { type: Array, default: () => [] },
  loading: { type: Boolean, default: false },
  configured: { type: Boolean, default: false },
})
const emit = defineEmits(['send', 'select-reference'])
const draft = ref('')
const messageList = ref(null)
const quickPrompts = [
  '什么是交通路网数据模型？',
  '轨迹数据有哪些关键技术？',
  '推荐一条学习路径',
]
watch(
  () => props.messages.length,
  async () => {
    await nextTick()
    if (messageList.value) {
      messageList.value.scrollTop = messageList.value.scrollHeight
    }
  },
)
function submit() {
  sendPrompt(draft.value)
}
function sendPrompt(value) {
  const question = value.trim()
  if (!question || props.loading) return
  emit('send', question)
  draft.value = ''
}
</script>
<style scoped>
.chat-panel {
  height: 100%;
  min-height: 0;
  border-left: 1px solid #e2e8f0;
  display: flex;
  flex-direction: column;
  background: #ffffff;
  overflow: hidden;
}
.chat-header {
  min-height: 70px;
  padding: 14px 16px;
  border-bottom: 1px solid #e7edf5;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 10px;
}
.chat-heading {
```

```
  min-width: 0;
  display: flex;
  align-items: center;
  gap: 9px;
}
.bot-icon {
  width: 34px;
  height: 34px;
  border-radius: 7px;
  display: grid;
  flex: 0 0 auto;
  place-items: center;
  color: #ffffff;
  background: #0f766e;
}
h2 {
  margin: 0 0 2px;
  color: #0f172a;
  font-size: 15px;
}
.chat-heading span {
  display: block;
  color: #64748b;
  font-size: 11px;
}
.status-dot {
  flex: 0 0 auto;
  padding: 4px 7px;
  border-radius: 999px;
  color: #9a3412;
  background: #fff7ed;
  font-size: 10px;
  font-weight: 700;
}
.status-dot.ready {
  color: #166534;
  background: #f0fdf4;
}
.message-list {
  flex: 1;
  min-height: 0;
  padding: 16px;
  overflow-y: auto;
  background: #f8fafc;
}
.message-row {
  display: flex;
  align-items: flex-start;
  gap: 8px;
}
.message-row + .message-row {
```

```
  margin-top: 14px;
}
.message-row.user {
  flex-direction: row-reverse;
}
.message-avatar {
  width: 26px;
  height: 26px;
  border-radius: 6px;
  display: grid;
  flex: 0 0 auto;
  place-items: center;
  color: #475569;
  background: #e2e8f0;
}
.assistant .message-avatar {
  color: #ffffff;
  background: #0f766e;
}
.message-bubble {
  max-width: calc(100% - 42px);
  padding: 9px 11px;
  border: 1px solid #e2e8f0;
  border-radius: 7px;
  color: #334155;
  background: #ffffff;
  font-size: 12px;
  line-height: 1.65;
  white-space: pre-wrap;
}
.message-content {
  min-width: 0;
  max-width: calc(100% - 42px);
}
.message-content .message-bubble {
  max-width: 100%;
}
.user .message-bubble {
  border-color: #bfdbfe;
  color: #1e3a8a;
  background: #eff6ff;
}
.message-references {
  margin-top: 6px;
  display: flex;
  flex-wrap: wrap;
  gap: 5px;
}
.message-references button {
  max-width: 100%;
  padding: 4px 7px;
```

```
  border: 1px solid #bfdbfe;
  border-radius: 5px;
  overflow: hidden;
  color: #1d4ed8;
  background: #eff6ff;
  cursor: pointer;
  font-size: 10px;
  text-overflow: ellipsis;
  white-space: nowrap;
}
.message-references button:hover {
  border-color: #60a5fa;
  background: #dbeafe;
}
.typing {
  display: flex;
  align-items: center;
  gap: 4px;
  min-width: 48px;
}
.typing span {
  width: 5px;
  height: 5px;
  border-radius: 50%;
  background: #64748b;
  animation: bounce 1s infinite ease-in-out;
}
.typing span:nth-child(2) {
  animation-delay: 0.15s;
}
.typing span:nth-child(3) {
  animation-delay: 0.3s;
}
.quick-prompts {
  padding: 10px 12px 0;
  display: flex;
  flex-wrap: wrap;
  gap: 6px;
}
.quick-prompts button {
  padding: 5px 8px;
  border: 1px solid #dbe4ef;
  border-radius: 5px;
  color: #475569;
  background: #ffffff;
  cursor: pointer;
  font-size: 10px;
}
.quick-prompts button:hover {
  border-color: #0f766e;
  color: #0f766e;
```

```
}
.chat-composer {
  padding: 10px 12px 12px;
  display: flex;
  align-items: flex-end;
  gap: 8px;
}
.chat-composer textarea {
  min-width: 0;
  flex: 1;
  padding: 9px 10px;
  border: 1px solid #d7dee8;
  border-radius: 6px;
  outline: 0;
  color: #0f172a;
  resize: none;
  background: #ffffff;
  font-size: 12px;
  line-height: 1.5;
}
.chat-composer textarea:focus {
  border-color: #0f766e;
  box-shadow: 0 0 0 3px rgba(15, 118, 110, 0.1);
}
.send-button {
  width: 38px;
  height: 38px;
  border: 0;
  border-radius: 6px;
  display: grid;
  flex: 0 0 auto;
  place-items: center;
  color: #ffffff;
  background: #0f766e;
  cursor: pointer;
}
.send-button:disabled {
  cursor: not-allowed;
  opacity: 0.45;
}
@keyframes bounce {
  0%,
  60%,
  100% {
    transform: translateY(0);
  }
  30% {
    transform: translateY(-3px);
  }
}
</style>
```